

## Scheduling

**Describir los objetivos que pueden tener las políticas de scheduling (fairness, carga del sistema, etc.).**

**Fairness:** Queremos que cada proceso reciba una dosis justa de CPU.

**Eficiencia:** Maximizar el uso del CPU.

**Throughput:** Maximizar el numero de procesos terminados por unidad de tiempo.

**Carga del sistema:** Minimizar la cantidad de procesos en *Ready*.

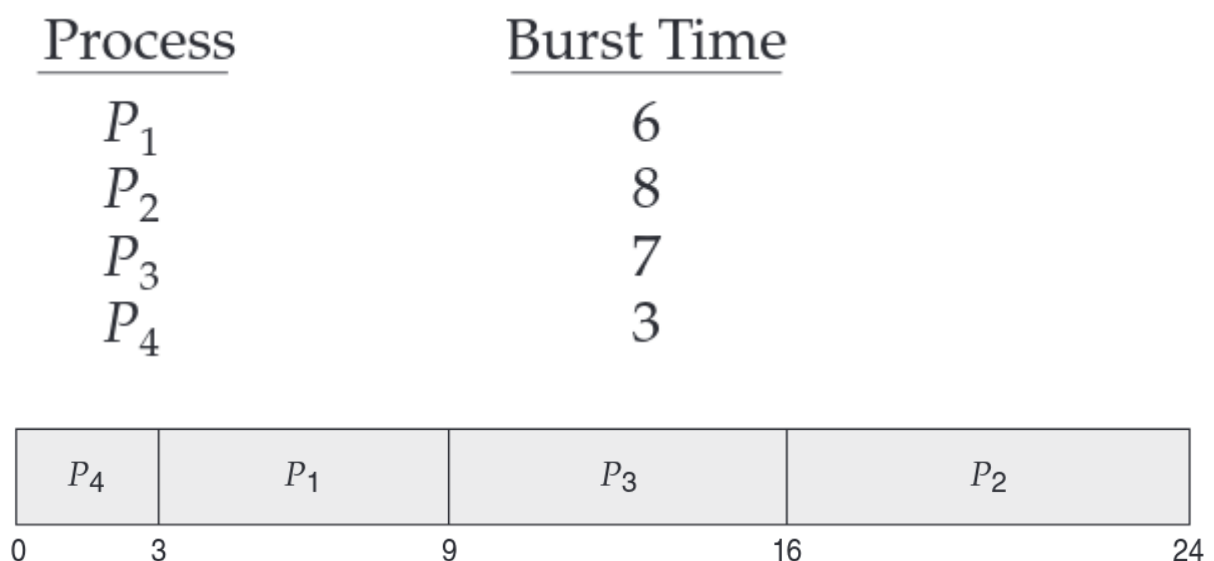
**Tiempo de respuesta:** Minimizar el tiempo de respuesta percibido por los usuarios.

**Latencia:** Minimizar el tiempo requerido para que un proceso de resultado.

**Tiempo de ejecucion:** Minimizar el tiempo *total* que le toma a un proceso ejecutar completamente.

**¿Qué objetivo prioriza SJF y por qué no se usa en la práctica?**

Esta orientado a maximizar el Throughput donde la idea es maximizar el numero de procesos terminaods por unidad de tiempo dandole prioridad a los proceso que tienen un tiempo estimado en terminar menor.



**P1:** Tiene waiting time de 3ms.

**P2:** Tiene waiting time de 16ms.

**P3:** Tiene waiting time de 9ms.

**P4:** Tiene waiting time de 0ms.

Por lo tanto el tiempo de espera promedio es de  $(3\text{ms} + 16\text{ms} + 9\text{ms} + 0\text{ms}) / 4 = 7\text{ms}$

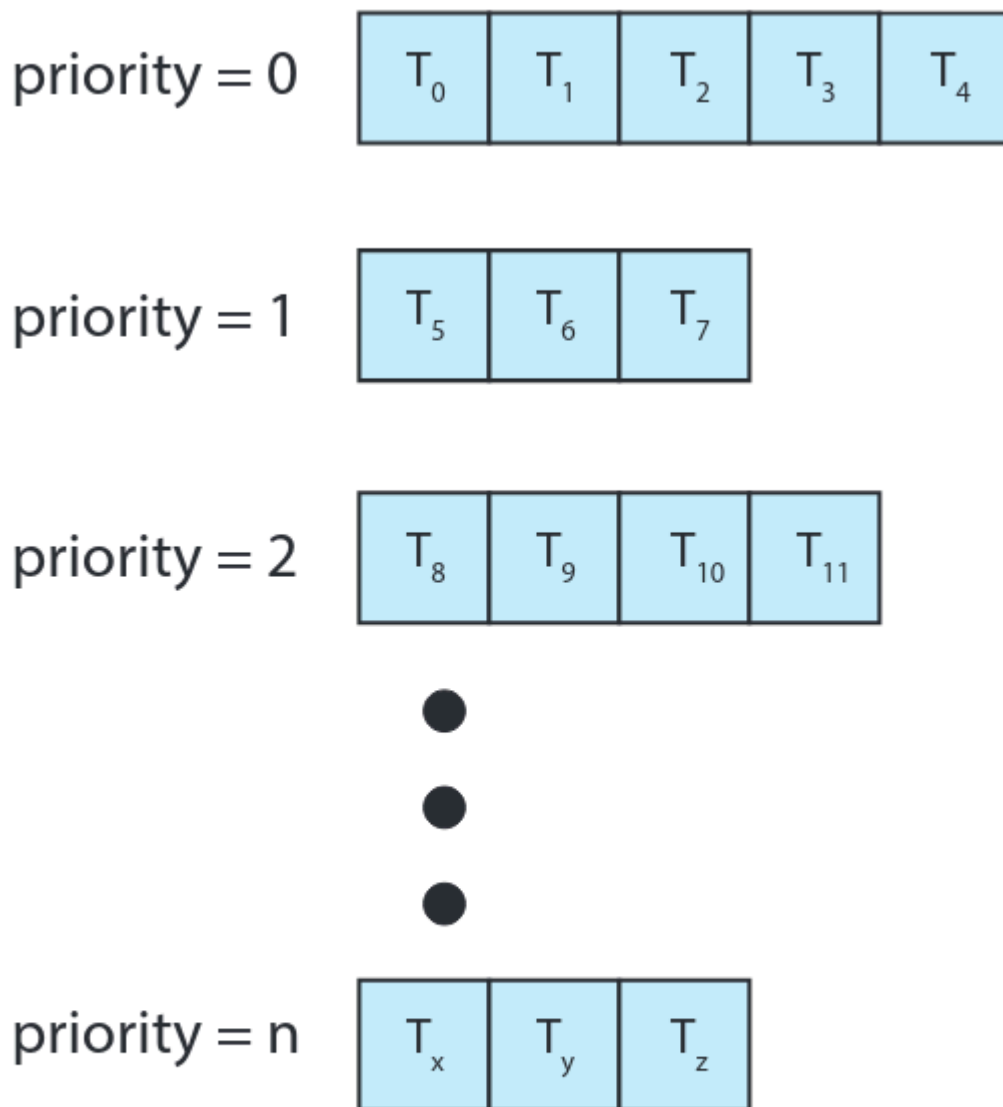
No se usa en la practica porque no hay manera de predecir cuanta rafaga de cpu va a ser necesitada por un proceso para completarse.

Puede ser con desalojo o voluntario. Si un proceso nuevo llega y necesita menor rafaga que lo restante del proceso actual entonces si es con desalojo se ejecuta el nuevo y si es voluntario se termina de ejecutar el viejo.

### **¿Cómo funciona el scheduling con múltiples colas?**

Se tienen colas separadas para cada nivel de prioridad. Luego para cada cola se usa round robin.

A cada proceso se le asigna una prioridad y este se mantiene en su misma cola durante toda su ejecución.



**Figure 5.7** Separate queues for each priority.

Hay procesos que necesitan mayor prioridad como los procesos interactivos.

Para las pilas multinivel tambien existen las *Multilevel-Feedback-Queue* que permite a los procesos moverse entre diferentes colas. Si un proceso usa demasiada rafaga de cpu se lo mueve a una cola mas baja dejando a los procesos de baja rafaga (interactivos, interfaz, entrada/salida) en las mayores prioridades.

Tambien, si un proceso esta mucho tiempo en una cola de baja prioridad se lo puede subir a una de mejor prioridad para evitar starvation.

For example, consider a multilevel feedback queue scheduler with three queues, numbered from 0 to 2 (Figure 5.9). The scheduler first executes all processes in queue 0. Only when queue 0 is empty will it execute processes in queue 1. Similarly, processes in queue 2 will be executed only if queues 0 and 1 are empty. A process that arrives for queue 1 will preempt a process in

queue 2. A process in queue 1 will in turn be preempted by a process arriving for queue 0.

- Un proceso entrante se lo pone en la *Queue 0*. A este se le da un quanta de 8ms.
- Si no termina en tiempo se lo mueve a la cola de la *Queue 1*.
- Si la *Queue 0* esta vacia se le da un quanta de 16ms al proceso en la cabeza de la *Queue 1*.
- Si no se completa su ejecucion, se lo manda a la *Queue 2*.
- Los procesos en la *Queue 2* solo corren si la *Queue 0* y la *Queue 1* estan vacias de forma *FIFO*.
- Si una *Queue* espera mucho tiempo en una cola de baja prioridad se la sube a una de mayor prioridad.

### ¿Hay algún problema con que las prioridades fueran fijas?

Si las prioridades son fijas puede ocurrir que un proceso tenga starvation porque siempre hay procesos con mayor prioridad ejecutando. En estos casos se rompe el fairness.

### Hablar sobre la afinidad de un procesador. ¿Qué información extra tenemos que tener en la PCB para mantener afinidad en un sistema multicore?

La idea es que el scheduler ponga a los procesos en *Ready* en procesadores donde estuvieron corriendo antes para aprovechar el cache y maximizar cache hits. Esto hace que tarde un poco mas en schedulear pero acelera bastante la ejecucion de los programas.

- Afinidad dura: A rajatabla, se hace siempre para todos los procesos.
- Afinidad blanda: Se intenta una vez, si no se puede, se envia a otro procesador.

En el PCB deberiamos tener un campo para marcar en que procesador fue la ultima ejecucion (o la de mas ejecuciones, los cache hits creo que dependen mas del tiempo desde que se ejecuto el proceso y no de la cantidad de veces).

### Explicar el problema de inversión de prioridades.